

REACT JS

```
import React, { useRef } from 'react'

const Hooks = () => {
  const inputref = useRef();

  const focusInput = ()=>{
    inputref.current.focus();
  }
  return (
    <div>

      <input type="text" ref={inputref} />

      <button onClick={()=> focusInput()}>Focus</button>
    </div>
  )
}

export default Hooks
```

```
import React, { useCallback, useMemo, useState } from "react";
import ChildMemo from "../ChildMemo";

const Memo = () => {
  const [count, setCount] = useState(0);
  const [number, setNumber] = useState(10);

  console.log("Memo component rendered");
  const handleClick = useCallback(() => {}, []);

  //use memo

  const calculation = useMemo(() => {
    console.log("calculation method");

    return number * 1000000000;
  }, [number]);

  return (
    <div>
```

```

    {calculation}

    <h1>fdsdfsdfass</h1>

    <h1>{count}</h1>

    <button onClick={() => setCount(count + 1)}>Add</button>

    <ChildMemo name="dsfadsf" handleClick={handleClick} />
  </div>
);
};

export default Memo;

```

```

import React from "react";

const ChildMemo = React.memo(({ name, handleClick }) => {
  console.log("Child memo component rendered");

  return (
    <div>
      <h1>Child memo</h1>
      <h2>{name}</h2>
    </div>
  );
});

export default ChildMemo;

```

```

import React, { useEffect, useState } from "react";
import axios from "axios";

const Component = ({ name, age }) => {
  // console.log(props);

  const [data, setData] = useState([]);

  const fetchData = async () => {
    const result = await axios.get(
      "https://jsonplaceholder.typicode.com/todos",
    );
  };

```

```

    // console.log(result.data);
    setData(result.data);
  };

  // side effect
  useEffect(() => {
    fetchData();
    // console.log("hello there");
  }, []);
  // no dependency => runs on every render
  // [] => runs only on mount
  // [count] => runs when that state change

  return (
    <div>
      <h1>jkhkjkhkj</h1>
      <h1>{name}</h1>
      <h2>{age}</h2>

      { /* list rendering */ }

      <div
        style={{
          display: "flex",
          justifyContent: "center",
          alignItems: "center",
          flexWrap: "wrap",
        }}
      >
        {data.map((ele) => {
          return (
            <div
              key={ele.id}
              style={{
                width: "200px",
                height: "200px",
                border: "2px solid black",
                color: "white",
                margin: "20px",
                padding: "20px",
                backgroundColor: "#333",
              }}
            >
              <h1>{ele.title}</h1>
            </div>
          );
        })}
      </div>
    );
  )}
</div>

```

```

    </div>
  );
};

export default Component;

```

```

import React, { useReducer, useState } from "react";

const UseReducer = () => {
  // const [count, setCount] = useState({count : 0});

  const reducer = (state, action) => {
    if (action.type === "increment") {
      console.log(action.color);

      return { count: state.count + 1 };
    } else if (action.type === "decrement") {
      return { count: state.count - 1 };
    } else if (action.type === "reset") {
      return { count : 0 }
    }
  };

  const [count, dispatch] = useReducer(reducer, { count: 1 });

  return (
    <div>
      <h1>useReducer</h1>
      <h1>{count.count}</h1>

      <button onClick={() => dispatch({ type: "increment", color: "Blue" })}>
        Add
      </button>

      <button onClick={() => dispatch({ type: "decrement"
    })}>Decrement</button>

      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
};

export default UseReducer;

```

```

import React, { createContext } from "react";

const UseContext = createContext(); // we created store

export const UserProvider = ({ children }) => {
  const user = {
    name: "FIIT",
    email: "Fiit@gmail.com",
  };

  return <UseContext.Provider value={user}>{children}</UseContext.Provider>;
};

export default UseContext;

// step 1: create store
// step 2 : create state and provider
// step 3 : usecontext we can access the store

```

```

import React, { useContext } from "react";
import UseContext from "../UseContext";

const Consumer = () => {
  const { name, email } = useContext(UseContext);
  console.log("consume component");

  return (
    <div>
      <h2>fasfadfas</h2>
      <h1>{name}</h1>
      <h1>{email}</h1>
    </div>
  );
};

export default Consumer;

```

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "../index.css";
import App from "../App.jsx";
import { UserProvider } from "../UseContext.jsx";

createRoot(document.getElementById("root")).render(
  // <StrictMode>

```

```

<UserProvider>
  <App />
</UserProvider>,
// </StrictMode>,
);

```

```

import React, { useState } from "react";

const Controlled = () => {
  const [name, setName] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(name);

    console.log("Form submitted");
  };
  return (
    <div>
      <h1>Controlled component</h1>

      <form onSubmit={(e) => handleSubmit(e)}>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Enter your name .."
        />
        <button type="submit">Submit</button>
      </form>
    </div>
  );
};

export default Controlled;

```

Theory

- **useRef** → Creates a mutable reference object. Commonly used to access DOM elements directly (e.g., focusing an input).
- **useMemo** → Memoizes expensive calculations so they don't re-run unnecessarily.
- **useCallback** → Memoizes functions to prevent re-creation on every render (useful when passing callbacks to child components).

- **React.memo** → Prevents child components from re-rendering unless props change.
- **useEffect** → Handles side effects like API calls, subscriptions, or DOM updates.
- **useReducer** → Alternative to useState for complex state logic. Uses a reducer function with state and action.
- **useContext** → Provides global state without prop drilling.
- **Controlled Components** → Form inputs controlled by React state.
- **Axios** → Promise-based HTTP client for API calls (get, post, put, delete).



Coding Round Questions

- **useRef** → Write a component with an input field and a button that focuses the input using useRef.
- **useMemo** → Implement a component that calculates a large number only when the dependency changes, using useMemo.
- **useCallback** → Create a parent-child component where the child is wrapped with React.memo and receives a memoized callback from the parent.
- **useReducer** → Build a counter using useReducer with actions: increment, decrement, reset.
- **useContext** → Create a context that provides name and email, and consume it in a child component.
- **Controlled component** → Implement a controlled form with an input field and a submit button.
- **Axios fetch** → Fetch data from an API using Axios and render it in a styled list.



Machine Coding Round Challenge

Problem: Todo App with React Hooks

Requirements:

1. Use **useState** to manage todos.
2. Use **useReducer** to handle actions: add, delete, reset.
3. Use **useContext** to provide global user info (name, email).
4. Use **useRef** to focus the input field when adding a new todo.
5. Use **useMemo** to calculate the total number of todos efficiently.
6. Use **Axios** to fetch initial todos from <https://jsonplaceholder.typicode.com/todos>.

Expected Flow:

- User sees initial todos fetched from API.
- User adds a new todo → input auto-focuses using useRef.
- Reducer updates state with add/delete/reset actions.
- Context displays user info globally.
- useMemo shows total todos count without recalculating unnecessarily.